Certified Distance Queries on Terrain Surfaces in Microseconds

Arslan Yamaletdinov
Affiliation

Abstract

Shortest-path distances on terrain surfaces underlie nearest-neighbour, range and route queries in geographic information systems, robotics and planning. Exact geodesic algorithms take seconds per pair on large terrains, and the distance oracles proposed for terrains answer faster but approximately, without saying how wrong a given answer is. We present the Certified Cut Index (CCI), a terrain distance oracle whose every answer is a certified interval: a query returns an estimate together with LB $\le d \le$ UB, where d is the exact surface distance, $(1-\varepsilon)d \le$ LB and UB $\le (1+\varepsilon)d$ (proved in exact arithmetic, and validated for our floating-point implementation against an exact solver). The interval adapts to the query, with a median width of 0.36% at $\varepsilon = 0.2$, and it makes the oracle a database operator: kNN and range queries are decided from the intervals, and only the candidates the intervals cannot separate are computed exactly. Every vertex stores exact geodesic distances to portals sampled on the straight cut lines of a kd hierarchy over the terrain. A window lemma lets us sample the cuts instead of storing them whole, an ellipse lemma makes every label an exact geodesic computed on a small sub-mesh, and the 1-Lipschitz label rows give the lower bound. A capped variant with certify-or-refine answers every query with a certificate or exactly, and on terrains of bounded slope and detour stores $O\big((k+\varepsilon^{-1}\log k)\, n \log n\big)$ labels. On the six terrains of the UP-Oracle paper with 0.5M faces, CCI answers in 0.5–2.1 μs on one core with a mean error of 0.001–0.03% in 0.36–0.66 GB, and answers exact 10-nearest-neighbour queries over 10,000 objects in 0.3–0.5 ms; EAR-Oracle needs 333 GB and 0.4–1 s per query for a mean error of 2.6–2.9%, and DGG, the most accurate competitor, 0.3–0.8 s per query with no per-query bound. Up to 5M faces the capped index grows like $n^{1.14}$. All 6.7 million answers we checked lie in their intervals. The same index answers queries between arbitrary surface points and extends to terrains with obstacles and, with an upper bound only, to weighted terrains.

1 Introduction

A terrain is usually stored as a triangulated irregular network (TIN): a triangulation of sampled points whose projection to the plane is a planar triangulation. Many queries on terrains are distance queries. A nearest-neighbour query asks for the closest objects to a location, a range query asks for all objects within a given distance, and both are routine in geographic information systems, in path planning for vehicles and robots, and in the analysis of disasters and military operations. The distance in these applications is the length of the shortest path on the surface, not the Euclidean or the projected one, which can be off by tens of percent in mountains.

Computing a single surface distance exactly is expensive. The classical algorithm of Mitchell, Mount and Papadimitriou (MMP) [23] and its practical successors [26, 28, 33] propagate a wavefront of windows over the mesh, and one query costs milliseconds on a few thousand vertices and seconds on a hundred thousand. For applications that ask many queries, the database community has proposed distance oracles for terrains: SE-Oracle [32], EAR-Oracle [16], RC-Oracle [35] and UP-Oracle [36]. They replace the surface by a Steiner graph or a point cloud and index it with well-separated pair decompositions or spanners. Their guarantees are a priori factors: $1 \pm \varepsilon$ against the exact distance between points of interest (SE-Oracle), $1+\varepsilon$ with an additional additive term for arbitrary points (EAR-Oracle), or a factor against a point-cloud graph rather than the surface (RC-Oracle). We measured mean errors of 0.8–3.5% and maximum errors up to 25% against exact geodesics, with indexes of hundreds of megabytes already at 10k vertices and of 333 GB at 0.5M faces. The most accurate practical method we found is not a terrain oracle at all: DGG [2] builds in minutes a sparse graph whose shortest paths approximate geodesics to a relative error of order ε when the vertices are evenly spread, and with $\varepsilon = 10^{-4}$ its mean error is about 0.004%. But DGG answers every query with a Dijkstra search on that graph, 0.3–6.9 s on terrains with 0.5–5M faces, and returns no bound for the individual answer. It is a better surface model, not an index; what it lacks is exactly what an index provides.

What none of these methods provides is a statement about a single answer. A query operator needs more than a good average. A kNN query compares the distances of many candidates, a range query compares each with a radius, and a planner may need to know that a route is no longer than a limit. With a point estimate of unknown error, every such decision is a guess near its boundary; with an interval LB $\le d \le$ UB that is guaranteed to contain the exact surface distance, most decisions can be made from the intervals alone, and the few that cannot are exactly the ones worth computing exactly (Figure 1). An oracle whose answer $\hat{d}$ is a path length with a guarantee $\hat{d} \le (1+\varepsilon)d$ implies the interval $[\hat{d}/(1+\varepsilon), \hat{d}]$, but only if the guarantee holds against the exact distance without an additive term, which only SE-Oracle's does, for points of interest, and its width is always ε. To decide a query, the interval must be narrow when it can be and known to be narrow.

Road networks faced the problem of fast distance queries and solved it with labelling indexes. H2H [24] and hierarchical cut labelling [12] let every vertex store its distances to the separator vertices of its ancestors, so that a query is a minimum over one separator. The same layout is behind Thorup's approximate oracles for planar graphs [29], which keep distances to $O(1/\varepsilon)$ portals on shortest-path separators, and it has been carried to weighted polyhedral surfaces through Steiner graphs [4, 11] and to mesh geodesics with compressed distance functions [15]. These indexes return a single value. On a terrain that value either rests on a discretised surface or comes with no bound that the query can check.

We propose the Certified Cut Index (CCI), a labelling index for terrain surfaces whose every answer is a certified interval. We keep the label layout and the minimum over one cut, and change

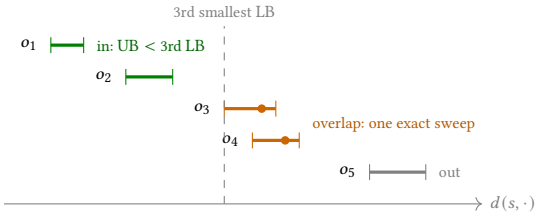


Figure 1: A certified 3-NN query from the intervals (schematic). o_1 and o_2 are in the answer because their upper bounds lie below the third smallest lower bound, and o_5 is out. Only o_3 and o_4 overlap, so one exact sweep from s decides between them (dots). With 10,000 objects at 0.5M faces, an exact 10-NN query needs such a sweep in 13–22% of the queries, for 0.3–0.5 objects on average, and takes 0.3–0.5 ms against seconds for a sweep over the terrain and 4–7 ms for a pruned on-the-fly search.

three things. First, the separators are straight axis-parallel cut lines of a kd hierarchy [6] over the chart of the terrain, not shortest paths. The portal argument of Thorup's oracles needs a shortest-path separator, so we replace it with a window lemma that bounds where a shortest path can cross a cut and what crossing far away costs; portals are then sampled at an arc gap g near the region and at geometrically growing gaps away from it. Second, every label is an exact geodesic distance on the continuous surface, computed on a small sub-mesh, rather than a graph, Steiner-graph or fitted value. Third, a straight cut makes the label rows 1-Lipschitz in arc length along the cut, and this gives a lower bound next to the upper bound. The resulting interval adapts to the query: at $\varepsilon = 0.2$ its median width is 0.36% uncapped and 2% with a cap of 64 portals per cut, far below ε. In summary (Theorems 4.4–4.6):

Theorem (informal). *Let T be a terrain, $\varepsilon \in (0,1)$ and $g > 0$, and let the labels be exact geodesic distances, as we compute them on unweighted terrains. In exact arithmetic, for every pair of vertices the index returns* $\mathrm{LB} \le d \le \mathrm{UB}$ *with* $(1-\varepsilon)d \le \mathrm{LB}$ *and* $\mathrm{UB} \le (1+\varepsilon)d$*, in time linear in the depth of the hierarchy and the number of portals of one cut. With at most k portals per cut inside the region, every answer satisfies* $\mathrm{UB} \le (1+\varepsilon)\mathrm{LB}$ *or is computed exactly, and on terrains of bounded slope and detour without obstacles the index stores* $O\big((k+\varepsilon^{-1}\log k)\, n \log n\big)$ *labels. Between arbitrary surface points,* $\mathrm{UB} \le (1+2\varepsilon)\mathrm{LB}$.

Our contributions are as follows: (i) a certified-interval distance oracle for terrains and its $(1+\varepsilon)$ guarantee, built on a window lemma that bounds where a shortest path can cross a straight cut, an ellipse lemma that makes labels computable exactly on small sub-meshes, and a lower bound read from the 1-Lipschitz label rows (Section 4); (ii) certified kNN and range queries that decide from the intervals and refine exactly only what they cannot decide, and a capped variant with certify-or-refine whose labels are near-linear in n under stated geometric assumptions (Sections 4.4, 4.5 and 5); (iii) engineering that keeps the bounds sound while making the index fast and compact: hybrid exact sweeps, closed-loop 8/16-bit label coding and a parallel build with bit-identical labels (Section 5); (iv) an extension to terrains with obstacles, with the same certificate, and to weighted terrains, with an upper bound only; and (v) an evaluation against every terrain oracle with available code, a mesh-geodesic index and a learned baseline, run on the same machine and the same queries, on terrains from 800 vertices to 5M faces (Section 6).

2 Related work

Exact and approximate geodesics. MMP [23] computes single-source geodesics exactly by propagating windows of constant unfolding over the mesh; Chen and Han [8] gave an $O(n^2)$ algorithm, and the practical implementations of Surazhsky et al. [28], ICH [33] and the triangle-oriented VTP [26] made exact geodesics routine. We use the first and the last to compute labels. Fast marching [20] and the heat method [9] approximate distances by solving PDEs on the mesh. The saddle vertex graph [37] and its successor DGG [2] precompute a sparse graph whose shortest paths approximate geodesics very well, but a query still runs Dijkstra on a graph with more edges than the mesh, and takes milliseconds.

Terrain distance oracles in databases. SE-Oracle [32] places Steiner points on the edges and builds a compressed partition tree with well-separated node pairs, answering POI-to-POI queries within a factor $1 \pm \varepsilon$ of the exact distance. EAR-Oracle [16] extends this to arbitrary points with highway nodes on an adaptive grid; its answer is at most $(1+\varepsilon)(d+2\delta)$, with an additive δ from the discretisation. RC-Oracle [35] targets point clouds, with a factor against the point-cloud graph, and UP-Oracle [36] terrains that change, both for a set of POIs. All of them compute their distances on a discretised surface and return a single value with an a priori guarantee, but no bound for the individual answer. EAR-Oracle is the closest in structure: it partitions the chart by a uniform grid and stores, for the vertices on box boundaries, approximate distances to the Steiner points inside the box, and answers a query by Dijkstra on these highway nodes. It has one level, approximate labels and an additive error term. Per-query lower and upper bounds on terrain distances are not new: Kaul et al. [17, 18] derive both, with a fixed ratio of about 0.9, by a search per pair, and earlier surface kNN methods [10, 27] filter candidates with Euclidean lower bounds and network upper bounds before an exact refinement. We follow the same filter-and-refine pattern, but our bounds are read from an index in a microsecond and their ratio is controlled by ε.

Oracles on polyhedral surfaces with theory. Aleksandrov et al. [4, 5] and Djidjev and Sommer [11] built $(1+\varepsilon)$ oracles for weighted polyhedral surfaces from a Steiner discretisation with geometrically placed points and shortest-path separators of the resulting graph. Djidjev and Sommer achieve near-linear space, $O(n\varepsilon^{-3/2}\log^2\frac{n}{\varepsilon}\log^2\frac{1}{\varepsilon})$, and query time $O(\varepsilon^{-1}\log\frac{1}{\varepsilon} + \log\log n)$. Near-linear space is therefore known in theory, and our uncapped index is asymptotically larger. As far as we know, neither line of work has an implementation, their constants include the Steiner density and the smallest face angle, and neither returns a per-query interval. Our geometric portal spacing is a relative of their geometrically placed Steiner points, applied along a cut instead of around vertices. In polygonal domains with obstacles, Thorup [30] and van der Laan et al. [31] use the same recursion with shortest-path separators of the free space and $O(1/\varepsilon)$ anchor points per vertex and separator.

Precomputed geodesics in geometry processing. The closest work to ours is SEP by Gotsman and Hormann [15]. It bisects a mesh recursively with separator curves, stores for every vertex its distances to the curves of its ancestors, restricted to each ancestor's patch, and answers a query by minimising over the shared curves; its conclusion suggests axis-parallel cuts as an alternative. Its distances are low-degree polynomial fits, the separators are sampled on a fixed grid, and pairs in the same leaf use the Euclidean distance. Its error bound holds only if every fit is accurate everywhere, which the method does not enforce, and it reports mean errors of 0.1–0.4% on meshes of up to 5k vertices. Because its distances are restricted to patches, every ancestor curve has to be scanned; our labels are distances on the whole surface, so the deepest separating cut suffices. We reimplemented SEP over exact VTP sweeps (its code is not public) and compare against it in Section 6.8. Landmark methods store distances to a few sample points: Xin et al. [34] interpolate them in constant time without a bound, and Burghard and Klein [7] return the triangle-inequality lower and upper bounds through the landmarks, whose relative width is not bounded.

Distance labels and portals on graphs. Planar graphs have small separators [22], and the approximate oracles of Klein [21], Thorup [29] and Kawarabayashi et al. [19] store distances to portals on shortest-path separators; Thorup's ε-covers keep $O(1/\varepsilon)$ portals per vertex and separator. Exact distance labels on planar graphs need $\Theta(\sqrt{n})$ entries per vertex in the worst case [13], which is the graph analogue of our $n^{1.5}$ total. On road networks, hub labels [1], pruned landmark labelling [3], H2H [24] and hierarchical cut labelling [12] answer exact queries in microseconds; the last two store, for every vertex, distances to the vertices of its ancestor separators or cuts. ALT [14] derives lower bounds from the triangle inequality through landmarks to guide a search. CCI transfers the layout of H2H to a continuous surface: cut lines instead of vertex separators, sampled portals instead of whole bags, exact geodesic labels, and a certified interval instead of an exact minimum. Its lower bound does not go through a landmark; it uses that a shortest path must cross the cut, and that the labels change by at most the arc length between neighbouring portals.

Learned distances. NeuroGF [38] fits a neural field to one mesh and answers a query with one forward pass, and GeGnn [25] learns a geodesic embedding with a graph neural network. Neither has an error bound, and a fixed capacity has to cover the whole surface.

3 Preliminaries

A terrain T is a triangulated height field: a triangulation of points (x_v, y_v, z_v) whose projection to (x, y) is a planar triangulation of a domain $U \subset \mathbb{R}^2$, the chart. We write n for the number of vertices. Each face f carries a cost $w_f \geq w_{\min} > 0$; an unweighted terrain has $w \equiv 1$. The cost $L(\gamma)$ of a curve γ on T is $\sum_f w_f$ times its 3-D length inside f, and $d(p, q)$ is the infimum over curves from p to q. Shortest paths exist by compactness. We assume that T is connected, and a query with $s = t$ returns LB = UB = 0. For a point p of T we write p also for its chart position and P for the point in space.

Two lower bounds are used throughout. Put $\lambda := w_{\min}$. On a face, the metric pulled back to the chart is $w_f^2(I + \nabla z \nabla z^\top) \succeq w_f^2 I$, and 3-D length dominates the chord, so for every curve γ from p to q

$$L(\gamma) \geq \lambda L_{\text{chart}}(\gamma) \geq \lambda |p - q|, \qquad L(\gamma) \geq \lambda |P - Q|. \quad (1)$$

The index has two parameters: the accuracy $\varepsilon \in (0, 1)$ and a portal gap $g > 0$, an arc length on the surface. They define the near radius $R := g/(\varepsilon\lambda)$. We consider vertex-to-vertex (V2V) queries and arbitrary-point (A2A) queries, and measure the relative error of an estimate $\hat{d}$ as $|\hat{d}/d - 1|$.

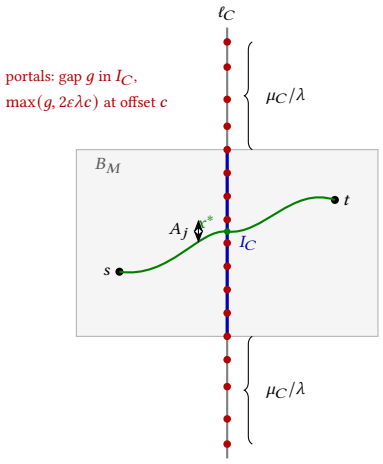


Figure 2: One cut of the index. The region M has bounding box B_M, and the cut line ℓ_C meets it in I_C. A shortest path from s to t must cross ℓ_C within the window, at most μ_C/λ from I_C (Lemma 4.1). Portals are placed at arc gap g inside the box and at gaps growing with the offset outside it. Both s and t store exact geodesic distances to all portals.

4 The Certified Cut Index

4.1 Cut hierarchy and windows

The index is built on a kd hierarchy of the vertices. A region M is a set of vertices with chart bounding box B_M. If the diagonal of B_M is at most R, the region is a leaf. Otherwise we cut it along the axis a of the larger extent at a value v strictly between the two middle member coordinates (if these are equal, between the nearest pair of distinct ones), chosen so that no vertex of T has a-coordinate v. The cut line is $\ell_C = \{x_a = v\}$, and the children are the members on its two sides. We write $I_C = \ell_C \cap B_M$.

A shortest path between two members of M on opposite sides of the cut must cross ℓ_C (by continuity of its chart projection), but it may leave the box first. The window bounds how far. Pick a hub $h \in M$ near the centre of the box and let $\mu_C = \max_{u \in M} \tilde{U}(h, u)$, where $\tilde{U}$ is the mesh-graph distance, an upper bound of d because a graph path is a curve on T.

Lemma 4.1 (window). *Let $s, t \in M$ lie on opposite sides of ℓ_C, let γ be a shortest path between them and $x^* \in \gamma \cap \ell_C$. Then $d(s, t) \geq 2\lambda \operatorname{dist}(x^*, I_C)$, and $\operatorname{dist}(x^*, I_C) \leq \mu_C/\lambda$.*

Proof. By (1), $d(s, t) \geq \lambda(|s - x^*| + |x^* - t|) \geq 2\lambda \operatorname{dist}(x^*, B_M)$, and for a point of ℓ_C the nearest point of B_M lies on I_C. On the other side, $d(s, t) \leq d(s, h) + d(h, t) \leq 2\mu_C$. □

The window $W_C = \{x \in \ell_C : \mathrm{dist}(x, I_C) \le \mu_C/\lambda\}$ therefore contains every crossing that matters. Its intersection with the surface is a union of segments, the runs, which we compute exactly from the faces the line crosses. On each run the lifted cut is a polyline, and we measure its cost $A(\cdot,\cdot)$ along it: arc length weighted by the face costs, which is plain arc length on an unweighted terrain. The cut is a curve on T, so $d(p,q) \le A(p,q)$ for two of its points.

4.2 Portals and labels

The portals P_C of a cut are points of the runs, sorted along the line, including the ends of every run and of I_C. For two consecutive portals of a run with arc distance A_j we require

$$A_j \le g \text{ if the interval lies in } I_C, \qquad A_j \le \max(g,\ 2\varepsilon\lambda c_j) \text{ otherwise,} \tag{2}$$

where c_j is the chart distance from the interval to I_C. Inside the box the portals are dense; outside it the gap grows linearly with the offset, which Lemma 4.1 allows because a path crossing at offset c is at least $2\lambda c$ long. The tails therefore hold only $O(\varepsilon^{-1}\log(\mu_C/\lambda R))$ portals.

Every member u of M stores the exact distances $\ell(u,p) = d(u,p)$ to all portals $p \in P_C$, and the rows of all cuts on the path from u's leaf to the root form its label. In addition, every vertex u stores a near table with the exact distances to the vertices $N_R(u)$ within chart distance R.

Exact labels on small sub-meshes. A label is a geodesic distance on the whole surface, but it can be computed on a small part of it.

Lemma 4.2 (ellipse). *If $d(x,u) \le D$, every point of a shortest path from x to u lies within chart distance $D/(2\lambda)$ of the segment $[x,u]$.*

Proof. By (1), a point z of the path has $|z-x|+|z-u| \le D/\lambda$, so it lies in the ellipse with foci x, u and major axis D/λ, whose points are within its semi-minor axis $\frac{1}{2}\sqrt{(D/\lambda)^2 - |x-u|^2} \le D/(2\lambda)$ of the segment. □

With D a graph upper bound through the hub, all label geodesics of a cut stay inside the bounding box of $M \cup P_C$ dilated by $D/(2\lambda)$. We take the faces that meet this box, insert the portals as vertices (on their faces' planes, so the surface does not change), and run one exact single-source sweep per portal on this sub-mesh. A geodesic of T inside the sub-mesh is a curve of the sub-mesh and conversely, so the sub-mesh distances are exact. The near tables are computed the same way, from one sweep per vertex on its own small sub-mesh.

4.3 Query and certificate

Algorithm 1 answers a query. If s and t are not near, they lie in different leaves, hence on opposite sides of the deepest cut C whose region contains both. Every f_j is the length of a path through p_j, so $\mathrm{UB} = \min_j f_j \ge d(s,t)$. The lower bound uses that a label row is 1-Lipschitz along the cut in arc length: $|\ell(u,p) - \ell(u,q)| \le A(p,q)$.

Algorithm 1 Query (s,t)

1: **if** $|s-t| \le R$ **then return** $d(s,t)$ from the near table of s
2: **end if**
3: $C \leftarrow$ the deepest cut whose region contains s and t
4: $f_j \leftarrow \ell(s,p_j) + \ell(t,p_j)$ for every portal $p_j \in P_C$
5: $\mathrm{UB} \leftarrow \min_j f_j$
6: $\mathrm{LB} \leftarrow \max\left(\lambda|S-T|,\ \min_j[\tfrac{1}{2}(f_j + f_{j+1}) - A_j]\right)$ ▷ consecutive portals of a run
7: $\hat{d} \leftarrow$ the interpolated minimum of f, clamped into $[\mathrm{LB}, \mathrm{UB}]$
8: **return** $(\hat{d}, \mathrm{LB}, \mathrm{UB})$

Lemma 4.3 (sandwich). $\mathrm{LB} \le d(s,t) \le \mathrm{UB}$ *for every pair.*

Proof. Near pairs are exact. Otherwise let x^* be a crossing of a shortest path, which lies on a run between consecutive portals p_j, p_{j+1} by Lemma 4.1, and let $a = A(p_j, x^*)$. The cut itself is a curve, so $d(x^*, p_j) \le a$, and $f_j \le d(s,x^*) + a + a + d(x^*,t) = d(s,t) + 2a$. Likewise $f_{j+1} \le d(s,t) + 2(A_j - a)$. Adding gives $\frac{1}{2}(f_j + f_{j+1}) - A_j \le d(s,t)$, and the chord term is a lower bound by (1). □

Theorem 4.4. *If the labels and near tables are exact, then in exact arithmetic, for every pair of vertices, $(1-\varepsilon)\,d(s,t) \le \mathrm{LB} \le d(s,t) \le \mathrm{UB} \le (1+\varepsilon)\,d(s,t)$, with equality when $|s-t| \le R$.*

Proof sketch. For a non-near pair, $d := d(s,t) \ge \lambda|s-t| > \lambda R = g/\varepsilon$ by (1). From the proof of Lemma 4.3, $\mathrm{UB} \le \min(f_j, f_{j+1}) \le d + A_j$. If the crossing interval lies in I_C, $A_j \le g < \varepsilon d$. Otherwise its offset satisfies $d \ge 2\lambda c_j$ by Lemma 4.1, and (2) gives $A_j \le \max(g, 2\varepsilon\lambda c_j) \le \varepsilon d$. The lower bound follows the same way: an interval term is at least $d - A_j$, and a tail interval with $A_j > \varepsilon d$ has both portals at offset more than $d/(2\lambda)$, where every f is at least $2\lambda c_j > d$. The full proof is in Appendix A. □

The certificate is not only a worst-case statement. UB/LB is known for every answer, and it is usually far below $1+\varepsilon$: the median interval width is 0.7–2% for vertex pairs in our experiments. The returned estimate is the interpolated minimum of the label sum, which in our experiments is 3–4 orders of magnitude more accurate than ε.

What is certified in floating point. The theorems assume exact labels and arithmetic. We compute labels in double precision, store them as float32 rounded up, and lower the lower bounds by the matching margin of 2^{-22}; a refined pair returns $d(1 \pm 10^{-12})$ around one sweep (10^{-9} between arbitrary points). These margins cover the storage rounding, not the rounding of the solvers, which we have not analysed, so we validate rather than prove: every answer in Section 6 is checked against an independent double-precision MMP sweep, and we call it certified and correct if $\mathrm{LB} \le d(1+10^{-9})$, $d \le \mathrm{UB}(1+10^{-9})$, and $\mathrm{UB} \le (1+\varepsilon)\mathrm{LB}$ or it was refined.

4.4 Capped portals and certify-or-refine

The top cuts are long and have $\Theta(\sqrt{n})$ portals each, and every vertex stores them, so the labels grow like $n^{1.5}$ in total. The capped variant puts at most k portals inside the box of each cut: its gap is $g_C = \max(g, A(I_C)/k)$, and its tails step by $\max(g_C, 2\varepsilon\lambda c)$. Lemma 4.3 does not use the gap, so the interval stays sound. The query accepts an answer with $\mathrm{UB} \le (1+\varepsilon)\mathrm{LB}$; otherwise it computes the distance exactly, with one sweep on the faces meeting $\mathrm{bbox}(s,t)$ dilated by $\mathrm{UB}/(2\lambda)$ (Lemma 4.2). For a cut C let r_C be the number of runs of its window, σ_C the largest cost per unit of chart length along it, and $\kappa_C = \mu_C/(\lambda|I_C|)$ the length of the window relative to the cut's span in the box.

Table 1: Size, query and build of the two variants. D: side of the chart; $\bar{\sigma}$: mean metric speed along the cuts; ρ: vertex density; K_C: portals of the answering cut. Except for the query, the entries assume bounded slope, detour and density, no obstacles, and a sweep over m faces in $O(m \log m)$ time.

	kd (Theorem 4.4)	capped (Theorem 4.5)
labels	$\Theta(n^{1.5})$; $\approx 3\bar{\sigma}D/g$ per vertex	$O((k+\varepsilon^{-1}\log k)\, n \log n)$
near tables	$\pi\rho g^2/(\varepsilon\lambda)^2$ per vertex	same
query	$O(\text{depth} + K_C)$	same, plus a rare exact sweep
build	$\Theta(n^{1.5}\log n)$	$O((k+\varepsilon^{-1}\log k)\, n \log^2 n)$

Theorem 4.5. *Under the assumptions of Theorem 4.4, every answer of the capped index satisfies* $\text{UB} \le (1+\varepsilon)\text{LB}$ *or is exact. A cut has at most* $k + O(r_C) + O\big(\frac{\sigma_C}{\varepsilon\lambda}(1+\log^+(\varepsilon k \kappa_C))\big)$ *portals, and the index stores* $\sum_u \sum_{C \ni u} |P_C|$ *labels and* $\sum_u |N_R(u)|$ *near-table entries, where* $N_R(u)$ *are the vertices within chart distance* R *of* u*. If* σ_C/λ, κ_C *and* r_C *are bounded, as on terrains of bounded slope and detour without obstacles, the labels hold* $O\big((k+\varepsilon^{-1}\log k)\, n \log n\big)$ *entries.*

Obstacles can add many runs, and clustered vertices large near tables; the theorem counts both. The acceptance test is stricter than the worst case of Theorem 4.4, which gives only $\text{UB}/\text{LB} \le (1+\varepsilon)/(1-\varepsilon)$. Building with $\delta = \varepsilon/(2+\varepsilon)$ in place of ε, so that $(1+\delta)/(1-\delta) = 1+\varepsilon$, would confine refines to pairs closer than g_C/δ across a capped cut. We build with ε and measure how often the refine runs (Section 6.8): with cap 64, for no random pair.

4.5 Arbitrary surface points

For a point p on face f with corners a_i and a point q on face f' with corners b_j, the straight segment inside a face is a path, so $d(p, a_i) \le w_f|P - A_i|$. The vertex certificates and the triangle inequality give

$$\text{UB}(p,q) = \min_{i,j}\ w_f|P-A_i| + \text{UB}(a_i,b_j) + w_{f'}|Q-B_j|,$$

$$\text{LB}(p,q) = \max\Big(\lambda|P-Q|,\ \max_{i,j}\ \text{LB}(a_i,b_j) - w_f|P-A_i| - w_{f'}|Q-B_j|\Big).$$

Pairs within R, and pairs whose interval is wider than $1+2\varepsilon$, are computed exactly by one sweep between the two points on an ellipse sub-mesh.

Theorem 4.6. *Every A2A answer satisfies* $\text{LB} \le d(p,q) \le \text{UB} \le (1+2\varepsilon)\text{LB}$.

4.6 Complexity

Table 1 summarises the costs. The build runs one exact sweep per portal, so it costs $\sum_C |P_C|\, S(m_C)$, where m_C is the number of faces of the sub-mesh of cut C and $S(m)$ the cost of one sweep. Lemma 4.2 bounds the region a sub-mesh covers, not its number of faces. If the dilation $D/(2\lambda)$ is proportional to the box, as with bounded slope and detour, and the vertex density is bounded, a cut at level ℓ of the hierarchy sweeps sub-meshes of $O(n/2^\ell)$ faces, and there are $O(\log n)$ levels; MMP and VTP sweep m faces in about $O(m \log m)$ time in practice, though not in the worst case. Under these assumptions the uncapped build is dominated by the top cuts, with $\Theta(\sqrt{n})$ portals each over $\Theta(n)$ faces, and the cap makes it near-linear. The near tables add one sweep per vertex on a sub-mesh of about ρR^2 faces. We state plainly that the asymptotic space of Djidjev and Sommer [11] is smaller than that of our uncapped index; our contribution is an implemented index that certifies every answer, with constants we measure.

5 Engineering

Hybrid exact sweeps. We use two exact solvers: Kirsanov's implementation of MMP [28] and VTP [26]. VTP is 2.6–4.1 times faster per full sweep, but its vertex queue does not make a popped vertex final, so it has no exact early stop. MMP processes windows in order of their minimum distance, and once that minimum exceeds the current value at every target no later window can improve any of them, so it stops early without changing a single distance. For each cut we choose MMP when the sub-mesh is large relative to the members' box, and VTP otherwise. VTP needs its per-sweep state reset and consistently oriented faces; it links its window lists from the first edge, so it also rejects sub-meshes that fall into several pieces, which we sweep piece by piece. On Gunnison10k the hybrid build takes 202 s, against 284 s with MMP alone and 215 s with VTP alone, with identical labels.

Closed-loop label coding. Labels are stored as float32 by default, rounded up so that they are never below the solver's double-precision value, and lower bounds subtract the matching relative margin. Most of the index is label rows, and neighbouring entries of a row differ by at most the arc gap between their portals (the rows are 1-Lipschitz). We therefore code a row in runs of deltas with 16 or 8 bits, anchored by a float32 value per run, and encode in a closed loop: each code is chosen from the decoded value of the previous entry, so the error does not accumulate. Every decoded value is at least the stored one, so the upper bound stays an upper bound; the lower bound subtracts the bounded decoding error, and the estimate is de-biased by half of it. A block scan over 8 portals at a time with four accumulators keeps the decoding cost low. The near tables are stored once per pair, in the row of the smaller vertex id, with varint id deltas; ratio-coded variants with 16 and 8 bits store $D = |PQ|(1 + c\,q)$ with the least code c that is not below the true distance.

Parallel build. The top cuts spread their portals over the threads and the remaining cuts go one per thread. The labels are bit-identical to a single-threaded build, two threads give a 2.0× speed-up, and the peak memory is about the coded index size because rows are coded as soon as a cut is done. A memory gate bounds the solvers alive at once. All timings in this paper use one thread.

Certified proximity queries. For a kNN or range query from s over a set of objects, the index gives an interval for each candidate. An object whose UB is below the k-th smallest LB is in the answer, one whose LB exceeds the k-th smallest UB is not, and only the ambiguous ones need an exact sweep from s, which we run once for all of them on an ellipse sub-mesh. Without that sweep, returning the k objects of smallest UB is a certified ε-approximate kNN: every returned object has $d \le (1+\varepsilon)d_k$, with d_k the true k-th distance. Likewise, the objects with $\text{UB} \le r$ form a range answer with no false positives that contains every object within $r/(1+\varepsilon)$.

Weighted terrains. Exact weighted geodesics have no practical algorithm, so on weighted terrains the labels are distances in a

Steiner graph with m points per edge, shared by all searches and searched with a stop once all members of a cut are settled. Graph paths are curves on the surface, so UB is still the cost of an actual path, but the labels are not exact, so the interval lower bound no longer holds and we keep only the chord bound $\lambda|S - T|$. The weighted index therefore returns an upper bound and an estimate, not a $(1 + \varepsilon)$ certificate, and the theorems of Section 4 apply to it only if exact weighted labels become available. Our weighted experiments are exploratory in this sense.

Obstacles. An obstacle removes the faces whose chart centroid lies inside it, so the surface gets a hole and paths go around it. The certificate needs no change. A hole only lengthens paths, so every chart lower bound still holds; μ_C is a graph distance on the holed mesh, so a detour widens the window by itself; a cut line broken by holes gives several runs, each bracketed by portals at its ends; and the near tables hold every pair within chart radius R with its exact distance, including pairs close in the chart but separated by a wall. One practical point matters. Where two holes touch at a single vertex the mesh is no longer a manifold, and there Kirsanov's MMP returned distances up to 5% too long, longer than a path in a Steiner graph. We therefore separate touching holes by keeping the largest fan of faces at such vertices; on the cleaned pilot meshes MMP and VTP agree to 10^{-15}.

Two defects in VTP.. On the cluster meshes, VTP as released returned distances up to 3% too long at a few vertices, longer than a path in a Steiner graph, while MMP never did. There are two causes, in its vertex queue and in a window-pruning rule. The queue is a `std::set` ordered by distance alone, so a vertex at exactly the distance of another queued vertex is silently dropped, or the wrong one is erased; such a vertex is never popped and never becomes a pseudo-source, and everything in its shadow gets a too-long distance. Ties are common on regular grids with integer heights, and rare on irregular meshes such as GunnisonForest, which is why earlier checks agreed. Ordering the queue by distance and vertex id, and removing a vertex before changing its key, fixes the queue. The remaining errors, up to 10^{-5}, come from the rule that drops one of two windows on an edge where their rays cross: it assumes the crossing lies on the surface, but each window is unfolded through its own chain of triangles, so the crossing may not exist. We no longer apply that rule. After both changes, MMP and VTP agree to $2 \cdot 10^{-11}$ or better on all 200 sources we checked on 28 meshes, and a VTP sweep takes 1.1–1.4 times as long as before on small meshes and 1.5–1.7 times at 0.5–2.5M faces. All results in Section 6.8 use the fixed VTP; the ground truth is MMP.

6 Experiments

Setup. The experiments on the small terrains run on an Apple M1 laptop (8 GB), and those on terrains with 0.5–5M faces on an HPC cluster (Section 6.8); every method runs on one thread. Our index is implemented in C++17. The ground truth is exact MMP in double precision. We use the four benchmark terrains of the SE-Oracle and EAR-Oracle papers (HorseMount, BigMount, HeadlightMount, RobinsonMountain, 793–3,696 vertices), Gunnison10k (9,610 vertices) and a 100k-vertex DEM, GunnisonForest100k. Our index and DGG answer the same 1,500 random vertex pairs per terrain (seed 7). EAR-Oracle and the other methods of its authors' harness answer 100 pairs with the points moved 5 cm into a face, because the harness cannot locate a point exactly at a vertex. The portal gap g is given in mean edge lengths, and we report ε and g per experiment.

Competitors. Every competitor runs with its authors' code: SE-Oracle, EAR-Oracle, K-Algo and the on-the-fly Steiner schemes in the harness of the EAR-Oracle authors [16], with a fix for weighted endpoints that changes their published weighted errors; UP-Oracle, WSPD-Oracle and RC-Oracle from the UP-Oracle code [36]; DGG [2] with a port to our platform; NeuroGF [38] with the authors' network and training, trained on exact distances and paths of our terrains; and exact MMP and VTP on the fly.

6.1 Main comparison

Table 2 and Figure 3 compare the methods.

Query time. CCI answers in 0.18–0.36 μs. EAR-Oracle takes 1.3–10.2 ms, so we are 7,000–28,000 times faster, and DGG, which runs Dijkstra on its graph, takes 1.4–22 ms. An exact VTP sweep per query costs 3.1–59 ms, and K-Algo and the on-the-fly Steiner schemes are slower still (Figure 3).

Accuracy. With $\varepsilon = 0.1$ our mean error is 0.0076–0.024% and the maximum 0.37–1.79%, while EAR-Oracle has a mean error of 3.1–3.5% and a maximum of 6.4–24.5%. Its nominal ε is relative to its Steiner graph, and the Steiner graph with 5 points per edge is itself about 3% longer than the surface. Increasing the number of Steiner points to 10 and 20 on HorseMount lowers EAR-Oracle's mean error only to 2.5% and 2.3%, at 541 and 683 MB. SE-Oracle reaches 0.81% on HorseMount but already needs 1.7 h of build and 942 MB at its 793 vertices, so on the larger terrains we compare against EAR-Oracle, its successor from the same group. DGG is the most accurate approximate method, with mean errors of about 0.0025%, but its queries are four orders of magnitude slower than ours and return no per-query bound.

Certificates. All 7,500 answers in Table 2 are certified and correct. The guaranteed bound is never violated, and it is about two times conservative in the worst case: with $\varepsilon = 0.1$ the largest UB error over all pairs is 0.9–3.4%, against the guaranteed 10%.

Size and build. The index is 1.3–37 MB, 19–145 times smaller than EAR-Oracle's, and builds 3–8 times faster. DGG's graph is two to three times larger than our index and builds faster.

6.2 Size against error

Figure 4 shows the size against the mean error over a sweep of ε and g. Our fronts lie entirely below and to the left of EAR-Oracle: on Gunnison10k the coarsest setting, $\varepsilon = 1$ at 21 MB, has a mean error of 0.31%, ten times lower than EAR-Oracle's best, at a 30th of its size. Pushing ε up to 3.2 with a fixed near radius brings our error to EAR-Oracle's level: 3.7% at 0.66 MB on HorseMount against EAR-Oracle's 3.1% at 189 MB, and 2.4% at 3.9 MB on RobinsonMountain against 3.5% at 672 MB. Every answer stays certified. On Gunnison10k, $\varepsilon = 0.05, 0.1, 0.2$ give 64, 37 and 24 MB with mean errors of 0.0024%, 0.0076% and 0.023%.

Table 2: Vertex-to-vertex queries, one thread. Ours: $\varepsilon = 0.1$, $g = 0.5$ edges, hybrid sweeps, float32 labels. Columns: build time / index size / query time / mean error / maximum error. Every answer of ours is certified and correct (LB $\le d \le$ UB on all pairs); none of the other methods returns a bound.

terrain	n	ours	EAR-Oracle (sp 5)	DGG ($\varepsilon = 10^{-4}$)	VTP
HorseMount	793	3.1 s / 1.3 MB / 0.18 μs / 0.024% / 1.16%	23.5 s / 189 MB / 1.3 ms / 3.10% / 24.5%	0.73 s / 4.4 MB / 1.4 ms / 0.0023% / 0.02%	3.1 ms
BigMount	1,440	10.4 s / 3.3 MB / 0.23 μs / 0.022% / 0.86%	52.4 s / 319 MB / 1.8 ms / 3.19% / 6.41%	1.5 s / 8.9 MB / 2.6 ms / 0.0025% / 0.02%	6.5 ms
HeadlightMount	2,470	29.1 s / 6.5 MB / 0.25 μs / 0.016% / 1.30%	107 s / 592 MB / 2.5 ms / 3.29% / 8.47%	2.9 s / 16.8 MB / 5.5 ms / 0.0026% / 0.02%	12.1 ms
RobinsonMountain	3,696	44.1 s / 10.7 MB / 0.28 μs / 0.020% / 1.79%	188 s / 672 MB / 3.8 ms / 3.50% / 8.60%	4.5 s / 25.3 MB / 7.9 ms / 0.0023% / 0.02%	18.5 ms
Gunnison10k	9,610	202 s / 37.4 MB / 0.36 μs / 0.0076% / 0.37%	672 s / 702 MB / 10.2 ms / 3.43% / 9.37%	13.2 s / 72.8 MB / 22.3 ms / 0.0028% / 0.01%	58.9 ms

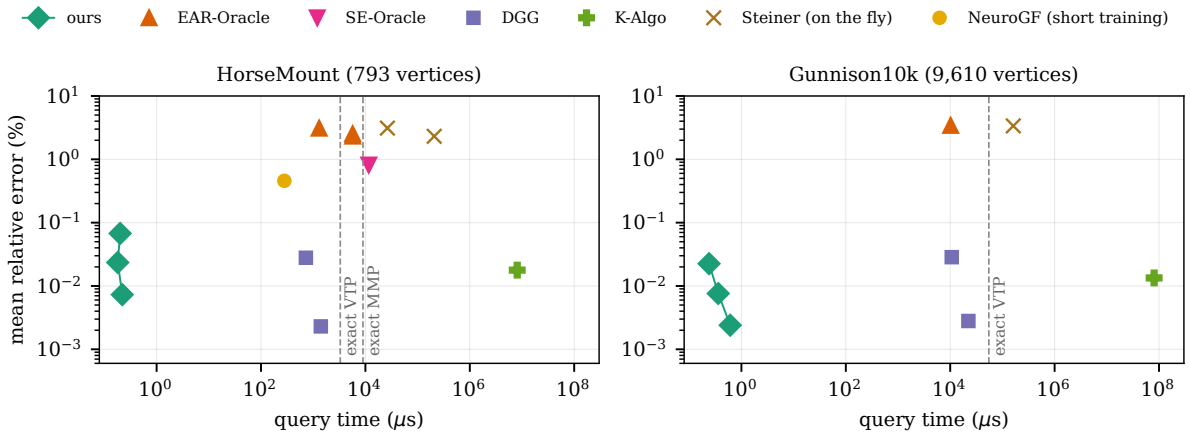


Figure 3: Query time against mean error for every method we ran, on the smallest benchmark terrain and on Gunnison10k. The three points of ours are $\varepsilon = 0.05, 0.1, 0.2$. Dashed lines mark the exact solvers, which have no error. NeuroGF was trained with a short budget on HorseMount only; its full-budget runs at 0.5M faces are in Section 6.8.

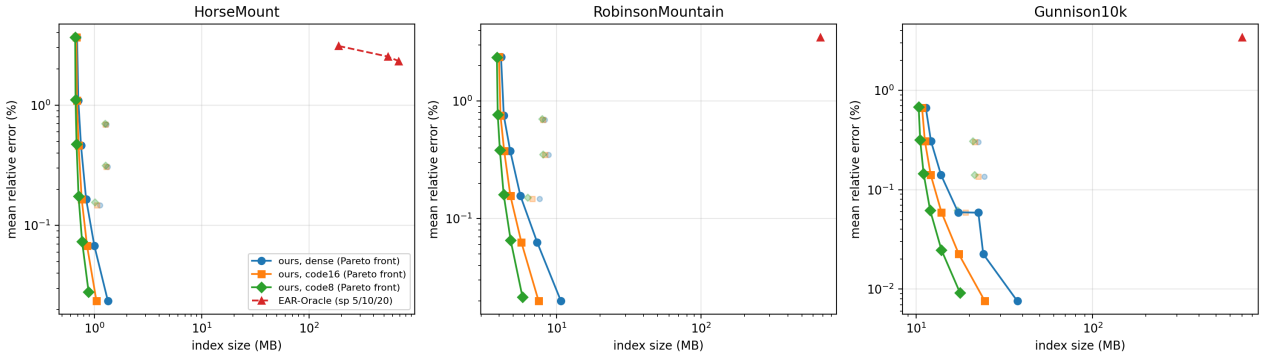


Figure 4: Index size against mean error. Each curve is the Pareto front of our index over ε and g for one label storage (float32, 16-bit and 8-bit codes); faded points are dominated settings. Triangles are EAR-Oracle with 5, 10 and 20 Steiner points per edge.

100k vertices. Table 3 gives the storage variants on Gunnison-Forest100k with $\varepsilon = 0.2$. The float32 index takes 534 MB, the 8-bit codes reduce it to 224 MB with the mean error going from 0.0069% to 0.0075%, and a cap of 50 portals per cut brings it to 177 MB at 0.041%. No pair needed refining. EAR-Oracle did not finish on this terrain on the 8 GB laptop; on the cluster it needs 93 GB and 5.4 h (Section 6.8). ε_c-covers in the style of Thorup [29], which keep for every vertex only the portals needed to reach the rest of the cut, are dominated by the 8-bit codes at every size we tried: at 100k, covers with $\varepsilon_c = 0.05$ take 230 MB for a mean error of 0.79%. The shifted hierarchy, three shifted quadtrees with a gap that grows with the cell size, is 1.4 times larger than the kd hierarchy at 100k vertices, as its constants predict; it becomes smaller from 1M faces (Section 6.8).

Table 3: Storage variants on GunnisonForest100k (100,511 vertices), $\varepsilon = 0.2$, one thread.

hierarchy	labels	size	mean error	query
kd	float32	534 MB	0.0069%	0.38 μs
kd	16-bit	330 MB	0.0069%	–
kd	8-bit	224 MB	0.0075%	1.3 μs
cap 100	8-bit	198 MB	0.017%	–
cap 50	float32	346 MB	0.039%	–
cap 50	8-bit	177 MB	0.041%	0.64 μs
kd	covers $\varepsilon_c = 0.05$	230 MB	0.79%	–

Table 4: Certified A2A queries, $\varepsilon = 0.1$, 100 random pairs of surface points per terrain.

terrain	query	mean err.	max err.	max UB err.
HorseMount	123 μs	0.11%	0.63%	6.2%
BigMount	111 μs	0.14%	2.69%	20.2% (10.2%)
HeadlightMount	89 μs	0.076%	0.77%	5.9%
RobinsonMountain	85 μs	0.11%	0.97%	11.4%
Gunnison10k	78 μs	0.034%	0.23%	3.3%

Growth with n. From 0.8k to 100k vertices the build time grows like $n^{1.45}$ and the index like $n^{1.32}$, close to the $n^{1.5}$ of Table 1; the 100k build takes 41–66 min on the laptop, depending on the load. The capped variant and the larger terrains are the subject of Section 6.8.

6.3 Arbitrary points

Table 4 gives the certified A2A queries. Every answer is sound, the mean error is 0.03–0.14%, and a query takes 78–123 μs, because the four corner pairs are looked up and a few pairs are refined. The refine step bounds the worst case: on BigMount one pair in 100 was refined, and the largest UB error fell from 20.2% to 10.2%. On the same kind of queries EAR-Oracle has a mean error of 4.9% and a maximum of 87% on HorseMount, at 4.4 ms per query.

6.4 Weighted terrains

On HorseMount with the face weights of the EAR-Oracle benchmark, our index with a Steiner graph of 5 points per edge has a mean error of 0.35%, a build time of 10.9 s, a size of 2.9 MB and a query time of 1.3 μs, measured against the shortest path found by any method. EAR-Oracle with 5 Steiner points has 2.42% at 25.9 s, 166 MB and 1.14 ms. Against a Steiner graph with 24 points per edge as the reference, our C++ index has a mean error of 0.38% with 5 points per edge and 0.23% with 10.

6.5 Obstacles

Table 5 gives a pilot on Gunnison10k with the obstacles of Figure 5. Every answer is certified and correct, uncapped and capped, over random, near, distance-band and A2A pairs, including the 208 pairs whose shortest path on the walls mesh is at least 20% longer than the straight line. The mean error is 0.03–0.04%, as without obstacles. On the walls mesh EAR-Oracle has a mean error of 3.59% and a maximum of 7.26%, with 36 ms per query and an index of 173 MB, and DGG ($\varepsilon = 10^{-3}$) has a mean error of 0.027% at 8.4 ms per query.

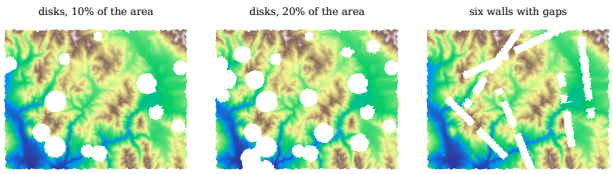


Figure 5: The obstacle meshes of the pilot on Gunnison10k. The faces inside the obstacles are removed.

Table 5: Obstacles on Gunnison10k (Figure 5), ours with $\varepsilon = 0.2$, $g = 1$ edge, cap 64. The last column counts the random pairs whose distance is at least 1.2 times their straight-line distance.

obstacles	build	mean err.	max UB err.	sound	pairs ≥ 1.2
disks, 10%	61 s	0.033%	4.3%	yes	5
disks, 20%	55 s	0.044%	1.1%	yes	20
six walls	131 s	0.032%	2.2%	yes	208

6.6 Proximity queries

On HorseMount, a certified kNN query with $k = 50$ over all vertices takes 27 μs in the approximate mode, against 0.5 ms for the exact answer from the index with refine and 0.85 ms for an on-the-fly search pruned by chart distances. On Gunnison10k with random objects the approximate mode answers kNN in 5 μs for $k = 1$, 20 μs for $k = 10$ and 165 μs for $k = 100$, and range queries in 2–7 μs. In all our runs its recall against the exact answer was at least 99.2%, and the guarantees of Section 5 held on every query. The exact mode needs one sweep only for the objects near the boundary of the answer; at 100k vertices it is 60 times faster than the pruned on-the-fly search for $k = 1$ and 1–2 times faster for large k, where that sweep dominates.

6.7 The POI setting

UP-Oracle and its baselines answer queries between a fixed set of points of interest. In that setting the simplest exact index is a $P \times P$ table built by one VTP sweep per POI. It is exact, sub-microsecond, and the cheapest to build for up to about 1,000 POIs on our terrains, which neither UP-Oracle nor our index beats. Our index in POI mode, which indexes only the POIs, becomes smaller than the table from about 1,000 POIs, and at 3,000 POIs on Gunnison10k it takes 3.1 MB against the table's 36 MB. Our index is designed for queries between all vertices and arbitrary points, where no table fits.

6.8 Terrains with 0.5–5M faces

The rest of the evaluation runs on the six terrains of the UP-Oracle paper [36] (TJ, SC, VS, AU, GI, LH) at 0.5M, 1M, 1.5M, 2M and 2.5M faces, and TJ also at 5M faces, on an HPC cluster node with two Intel Xeon Gold 6248R processors and 768 GB of memory. Every method runs on one thread, except the parallel builds where stated and NeuroGF, which trains and is queried on one V100 GPU (we also report its single-thread CPU query time). The ground truth is exact MMP in double precision. Each terrain has 2,000 random

vertex pairs (200 sources with 10 targets), 2,000 pairs closer than the near radius, 300 pairs in each of up to ten distance bands, and 200 pairs of arbitrary surface points. All methods answer the same 2,000 random pairs, except DGG, whose searches take up to seconds and which answers the first 200 of them. Ours uses $\varepsilon = 0.2$ and $g = 1$ edge unless stated, so its index is smaller than in Table 2. Over all runs of our index on unweighted terrains on the cluster, we checked 6.67 million answers against the ground truth: about 0.29 million distinct query pairs, answered by every index variant and storage format. Every answer lies in its interval, up to the tolerance of Section 4. Our query time is the lookup: the mean over a query set, best of five passes. Where the capped index refines, we also give the end-to-end mean, timed per query in one pass with refines included. Sizes are the byte counts of the stored structures for our index and SEP, the graph file for DGG, and the growth of EAR-Oracle's resident memory over its build, as its harness reports it, so they compare in order of magnitude only. The nodes are shared with other users' jobs, and two runs of the same build can differ in time by up to 1.6 times (11.7 and 17.9 h for the uncapped TJ index at 0.5M faces); where we have repeats, we report the shorter.

Main comparison at 0.5M faces. Tables 6 and 7 and Figure 6 give the results. Uncapped, our index answers in 1.0–1.9 μs with a mean error of 0.0011–0.0016% and a maximum of 0.15–0.30%, in 1.9–2.1 GB of float32 labels or 0.60–0.66 GB of 8-bit codes, built in 10–19 h on one thread. With a cap of 64 portals per cut it takes 355–366 MB, answers in 0.5–1.5 μs with a mean error of 0.014–0.029%, and builds in 7–11 h; on TJ and GI the exact refine was needed for none of the random pairs, and for one pair each over all query sets. EAR-Oracle with its default of 5 Steiner points per edge needs 333 GB and 17–21 h, answers in 0.4–1.0 s, and has a mean error of 2.6–2.9% and a maximum of 8.5–9.8%. With 7 points it needs 468 GB and 35 h and still has 1.9%. Its peak memory follows from its design: every leaf box stores the distances between its boundary vertices and all Steiner points inside it, both ways, in a balanced tree, which grows like the number of Steiner points times $n^{1.5}$ for a fixed number of boxes. On the authors' own GunnisonForest (100,511 vertices) it needs 93 GB and 5.4 h, with a mean error of 2.9%; our index on the same terrain takes 480 MB, 1.0 h and 0.0077%. So our index is 540–930 times smaller, answers five to six orders of magnitude faster, and its mean error is three orders of magnitude lower.

DGG is the strongest competitor in accuracy and build time: its graph builds in minutes, and with $\varepsilon = 10^{-4}$ its mean error is 0.004%, comparable to ours. But a DGG query is a Dijkstra search on a graph of 0.8–2.4 GB and takes 0.28–0.82 s, and it returns no per-query bound. At the same size our index is more accurate: with 8-bit labels the uncapped index takes 0.62 GB on TJ for a mean error of 0.0021%, against DGG's 0.0040% at 2.4 GB (Figure 6). We see DGG and our index as answering different needs: DGG is the better choice for a handful of queries on a terrain that will not be queried again, ours when the terrain is queried many times, needs answers in microseconds, or needs a bound on each answer. NeuroGF is the smallest index by far, 0.25 MB of network weights, but its mean error is 0.2–0.6% and single pairs are off by 4–13%, with no indication of which. We checked that this is not an artefact of our setup. Distance-to-surface targets measured exactly instead of against a subsampled surface give the same accuracy (0.41% against 0.40% on TJ), and 800 epochs instead of their 400 lower the mean error by 11–15% and the maximum from 13.4% to 9.1%, at twice the training time. An exact VTP sweep takes 6.3–6.6 s per query. SEP [15], which we reimplemented with exact VTP sweeps, builds in 2.7–3.1 h into 228–236 MB (110–114 doubles per vertex) and answers in 6.5–8.7 μs with a mean error of 0.34–0.37%, close to the 0.1–0.4% its authors report on meshes of up to 5k vertices. It builds faster than our index and is smaller than any index but NeuroGF's, but its errors go both ways, from −2.8% to +2.4%, and nothing in an answer says which pairs are off. At about its size our index is more accurate and faster: with $\varepsilon = 0.8$, $g = 4$ and 8-bit labels it takes 265 MB, builds in 3.3–3.6 h and answers in 0.6 μs with a mean error of 0.016–0.020%, about 20 times lower, and a certified interval for every answer (Figure 6). Same-leaf pairs, which SEP answers with the Euclidean distance, did not occur among the 2,000 pairs.

When the build pays off. Our build takes hours where DGG's takes minutes, but it is amortised quickly. With cap 64 at 0.5M faces it pays for itself after 3,700–4,500 queries compared with answering every query by an exact VTP sweep, and after 30,000–34,000 queries compared with DGG ($\varepsilon = 10^{-4}$, including DGG's own build); the uncapped index after 6,400–7,100 and 51,000–54,000 queries. At 2.5M faces the capped index breaks even after 7,800–8,100 queries against VTP and 49,000 against DGG; the uncapped index built on one thread (110–125 h) after 16,600–19,600 and 106,000, and built on 32 threads (15.7 h) after 2,400 and 14,800. At 5M faces, where a VTP sweep takes 74 s and a DGG query 6.9 s, the capped index built on 32 threads (12.0 h) breaks even after 590 and 5,700 queries. Against EAR-Oracle there is no break-even point: its build is longer and each of its queries slower. A single kNN query over 10,000 objects asks for up to 10,000 distances, so one batch of proximity queries, or the queries of a route planner over a day, already exceeds these counts.

Error by distance. On TJ, pairs closer than the near radius are answered exactly from the near tables. The error peaks just beyond the near radius, with a mean of 0.13% and a maximum of 1.8% in the first distance band, and falls with distance to a maximum of 0.0007% in the last band: the window lemma leaves the most slack where the two points are close to the cut.

Arbitrary points and weights. A2A queries between arbitrary surface points take 10–18 μs uncapped, with a mean error of 0.002–0.006%, and 4–7 μs with cap 64 at 0.016–0.037%, all certified. EAR-Oracle has a mean error of 2.6% and 2.9% on the same A2A pairs of TJ and GI, at 237 and 389 ms per query, and NeuroGF 0.37% on TJ. On the weighted versions of TJ and GI, with the face weights of the EAR-Oracle benchmark, our index with 5 Steiner points per edge differs from a reference Steiner graph with 12 points by 0.10% on average (maximum 0.6–0.9%), takes 1.6 GB with 8-bit labels, builds in 9.8 h and answers in 2.1–2.2 μs. EAR-Oracle differs from the same reference by 1.5–1.8%, with 333 GB, 23 h and 0.35–0.63 s per query. For many pairs our upper bound is shorter than the reference, so the reference is not exact and these are differences rather than errors; as in Section 5, the weighted index certifies only the upper bound.

Scaling to 5M faces. Table 8 and Figure 7 follow TJ and GI from 0.5M to 2.5M faces (1.25M vertices), and TJ further to 5M faces.

Table 6: Our index on the six terrains at 0.5M faces (251,001 vertices), $\varepsilon = 0.2$, $g = 1$, one thread. Errors are over the 2,000 random vertex pairs; the maximum UB error is the largest $(\mathrm{UB} - d)/d$. Uncapped: float32 / 8-bit size, errors and query time of float32 labels. Cap 64: 8-bit labels. Every answer is certified and correct.

	uncapped					cap 64					NeuroGF	
terrain	build	size	query	mean err.	max err.	build	size	query	mean err.	max err.	mean	max
TJ	11.7 h	1.99 / 0.62 GB	1.47 μs	0.0012%	0.27%	6.8 h	358 MB	0.55 μs	0.021%	1.16%	0.40%	13.4%
SC	13.2 h	1.98 / 0.61 GB	1.45 μs	0.0011%	0.21%	6.8 h	357 MB	0.54 μs	0.021%	1.72%	0.39%	6.5%
VS	19.1 h	2.14 / 0.66 GB	1.26 μs	0.0012%	0.19%	11.4 h	366 MB	0.96 μs	0.029%	3.15%	0.56%	11.6%
AU	10.4 h	1.93 / 0.60 GB	1.29 μs	0.0011%	0.15%	6.9 h	355 MB	0.54 μs	0.014%	0.67%	0.21%	4.1%
GI	12.3 h	2.02 / 0.62 GB	1.03 μs	0.0016%	0.21%	7.8 h	359 MB	1.54 μs	0.029%	4.52%	0.44%	11.4%
LH	12.6 h	1.95 / 0.61 GB	1.87 μs	0.0015%	0.30%	7.3 h	356 MB	0.53 μs	0.020%	1.84%	0.34%	9.0%

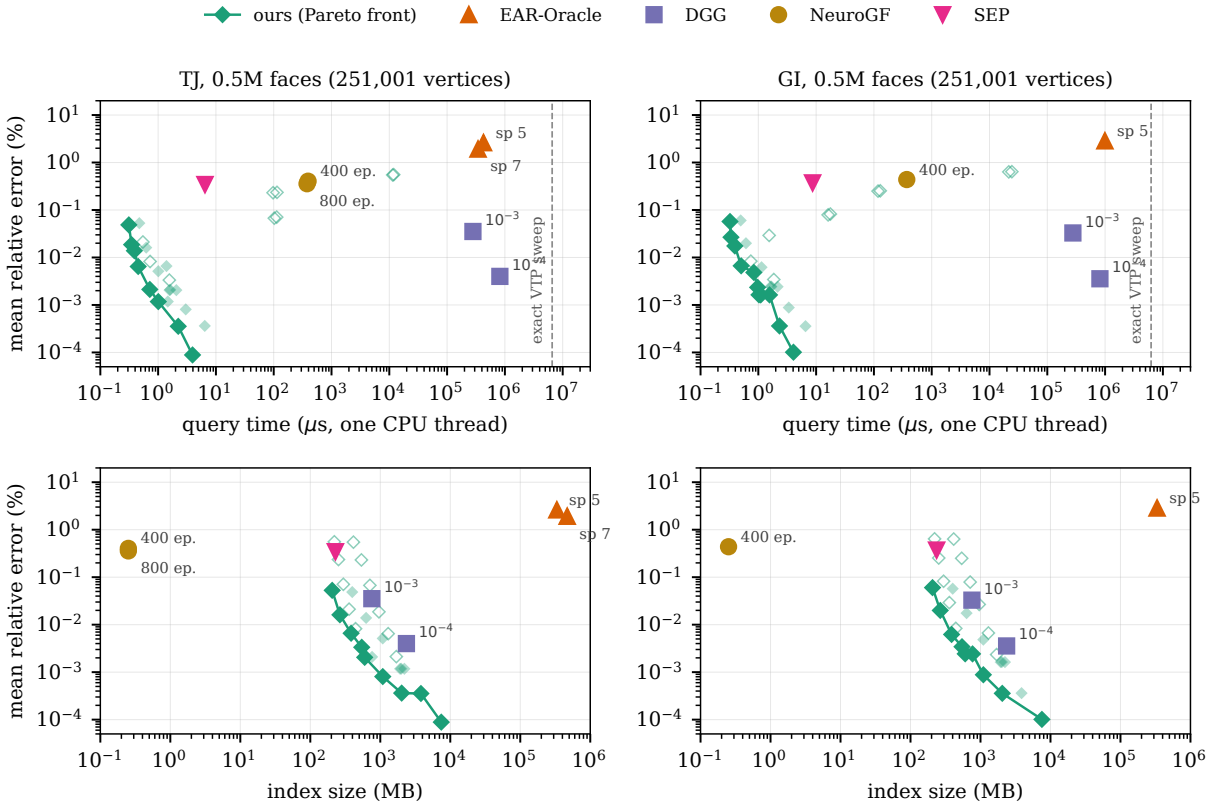


Figure 6: Query time (top) and index size (bottom) against mean error on TJ and GI at 0.5M faces. Our points cover the uncapped index, $\varepsilon = 0.1$, caps $k = 8, \ldots, 256$ and a sweep of ε and g, each with float32 and 8-bit labels; hollow points are capped and the line is the Pareto front. Our query time is the lookup, or the end-to-end mean with the exact refines where they ran (caps 8–32), so every point's time is that of the answers whose error it shows.

The uncapped index stores labels growing like $n^{1.48}$, the $\Theta(n^{1.5})$ of Table 1, and reaches 23 GB at 1.25M vertices; its build grows like $n^{1.45}$–$n^{1.47}$ and takes 96–110 h on one thread at 1M vertices and 110–125 h at 1.25M vertices. With cap 64 the labels grow like $n^{1.14}$, the index takes 6.0 GB (2.1 GB with 8-bit codes) at 1.25M vertices, and the build grows like $n^{1.2}$ and takes 51–52 h. The query time stays at 0.3–1.2 μs capped and grows to 3.0–3.3 μs at 1M vertices uncapped, while the mean error does not grow: it falls from 0.0012–0.0016% to 0.0007–0.0008% uncapped and stays at 0.019–0.027% capped. All answers at every size are certified and correct. The parallel build of the uncapped index on the 2.5M-face TJ takes 24.6, 16.0 and 15.7 h with 8, 16 and 32 threads, with identical labels; the speed-up flattens beyond 16 threads, where the top cuts, whose sweeps run on large sub-meshes, dominate.

To see whether the capped index keeps scaling, we resampled the 2.5M-face TJ grid to 5M faces (2.51M vertices) on the same

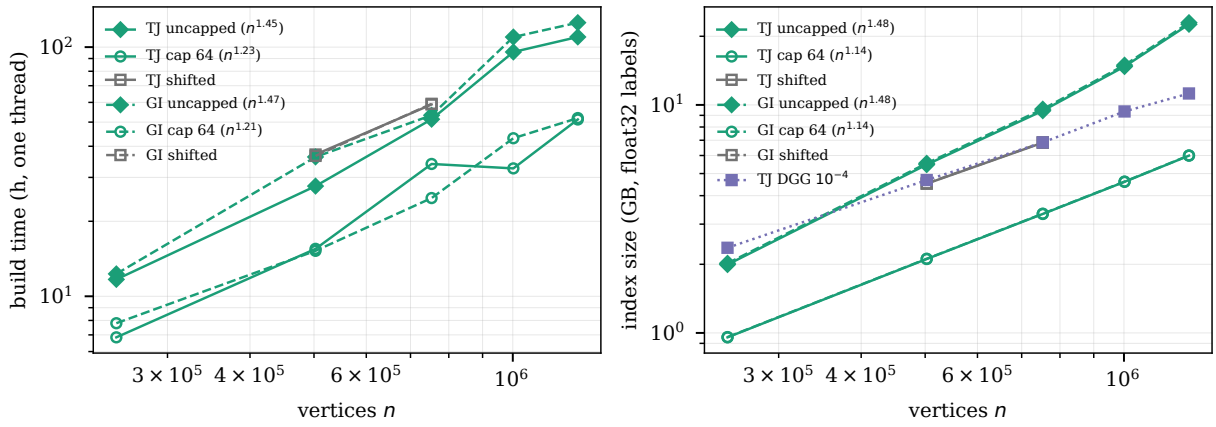


Figure 7: Build time (one thread) and index size from 0.5M to 2.5M faces on TJ and GI, with the exponents of a power-law fit, and DGG's graph for comparison.

Table 7: All methods on TJ and GI at 0.5M faces, vertex pairs, one CPU thread. Ours with 8-bit labels. NeuroGF trains on a V100 GPU; its query time is on one CPU thread (0.3–0.4 μs batched on the GPU). Only ours returns a certified bound.

	method	build	size	query	mean	max
TJ	ours, uncapped	11.7 h	0.62 GB	2.1 μs	0.0021%	0.28%
	ours, cap 64	6.8 h	0.36 GB	0.55 μs	0.021%	1.16%
	ours, $\varepsilon = 0.1$	29.4 h	1.1 GB	3.0 μs	0.0008%	0.055%
	EAR-Oracle, sp 5	17.6 h	333 GB	426 ms	2.61%	9.75%
	EAR-Oracle, sp 7	35.4 h	468 GB	343 ms	1.91%	6.90%
	DGG, 10^{-3}	2 min	0.76 GB	280 ms	0.035%	0.16%
	DGG, 10^{-4}	7 min	2.4 GB	819 ms	0.0040%	0.036%
	NeuroGF (800 ep.)	6.9 h	0.25 MB	0.39 ms	0.36%	9.1%
	SEP (our reimpl.)	3.1 h	228 MB	6.5 μs	0.34%	2.1%
	exact VTP	–	–	6.6 s	0	0
GI	ours, uncapped	12.3 h	0.62 GB	1.6 μs	0.0025%	0.21%
	ours, cap 64	7.8 h	0.36 GB	1.5 μs	0.029%	4.52%
	ours, $\varepsilon = 0.1$	27.5 h	1.1 GB	3.4 μs	0.0009%	0.077%
	EAR-Oracle, sp 5	21.3 h	333 GB	993 ms	2.85%	8.46%
	DGG, 10^{-4}	7 min	2.4 GB	814 ms	0.0036%	0.015%
	NeuroGF	3.5 h	0.25 MB	0.37 ms	0.44%	11.4%
	SEP (our reimpl.)	2.7 h	236 MB	8.7 μs	0.37%	2.8%
	exact VTP	–	–	6.3 s	0	0

surface, adding no relief. Built on 32 threads in 12.0 h, the capped index takes 12.9 GB, or 4.7 GB with 8-bit codes, 2.15 times its size at 2.5M faces, in line with $n^{1.1}$. It answers in 0.34 μs (0.52 μs with 8-bit codes) with a mean error of 0.019% and a maximum of 1.3%, without a single exact refine, and all answers, over every query set, are certified and correct. On this terrain DGG with $\varepsilon = 10^{-4}$ builds a 23.6 GB graph in 1.1 h and answers in 6.9 s, and an exact VTP sweep takes 74 s. The shifted hierarchy, 1.4 times larger than the kd hierarchy at 100k vertices, is smaller from 1M faces on (4.5 against 5.5 GB at 1M, 6.8 against 9.4 GB at 1.5M), the crossover its constants predict, but its mean error is 0.08–0.09% and it builds more slowly, so cap 64 (2.1 and 3.3 GB at 0.02%) dominates it. DGG's graph grows

Table 8: Scaling on TJ, one thread: uncapped index (float32) and cap 64 (float32; 8-bit in brackets), against DGG ($\varepsilon = 10^{-4}$) and an exact VTP sweep per query. Mean errors are over 2,000 random vertex pairs. [b]The 5M-face terrain, resampled from the 2.5M-face grid on the same surface; built on 32 threads.

	uncapped			cap 64		DGG	VTP
n	build	size	mean	build	size	size / query	query
251k	11.7 h	2.0 GB	0.0012%	6.8 h	0.96 (0.36) GB	2.4 GB / 0.8 s	6.6 s
503k	27.7 h	5.5 GB	0.0009%	15.5 h	2.1 (0.78) GB	4.7 GB / 1.6 s	11.8 s
753k	51.4 h	9.4 GB	0.0009%	34.0 h	3.3 (1.23) GB	6.9 GB / 2.1 s	16.6 s
1.00M	95.6 h	14.7 GB	0.0007%	32.6 h	4.6 (1.66) GB	9.4 GB / 3.3 s	24.2 s
1.25M	109.8 h	22.6 GB	0.0006%	51.2 h	6.0 (2.14) GB	11.2 GB / 3.7 s	23.8 s
2.51M[b]	–	–	–	12.0 h[b]	12.9 (4.65) GB	23.6 GB / 6.9 s	74 s

linearly and is smaller than our uncapped index from 1M faces, but larger than the capped one at every size, and its queries slow down from 0.8 to 3.7 s. From its memory growth EAR-Oracle would need about 0.9 TB at 1M faces, more than the 768 GB of a node.

Ablations. On TJ and GI at 0.5M faces the cap trades size for error smoothly: caps of 8, 16, 32, 64, 128 and 256 portals give 413 MB to 1.70 GB (float32), with mean errors from 0.55–0.64% down to 0.002%. The exact refine runs for 128–137, 34–36, 9–12, 1 and 0 of the 6,621 pairs with caps 8, 16, 32, 64 and 128. With cap 8 it runs for 1.0–1.3% of the random pairs at 1.1–1.7 s each, so the end-to-end mean is 11–22 ms against a 0.37 μs lookup, a cost the 99th percentile hides on TJ (12 μs); with cap 64 (8-bit) it is 0.60 and 1.54 μs on TJ and GI, against lookups of 0.55 and 1.54 μs. Thorup-style ε_c-covers are dominated: with $\varepsilon_c = 0.05$ they take 483 MB for a mean error of 0.82%, worse than cap 8 at 413 MB, and with $\varepsilon_c = 0.01$ 1.13 GB for 0.082%, against 0.0021% for the uncapped 8-bit index at 0.62 GB. For the label sweeps, MMP alone takes 19–24 h with 3 GB of memory, and VTP alone 18–27 h with 300 GB. The hybrid takes 13–18 h with 30 GB, and 18 h with 75 GB when it prefers VTP more often, so it is 1.4–1.5 times faster than either solver alone with a tenth of VTP's memory.

Table 9: Obstacles on TJ and GI at 0.5M faces (disks covering 5, 10 and 20% of the area, and six walls with gaps), vertex pairs. Ours: $\varepsilon = 0.2$, $g = 1$, float32. The detour column counts the random pairs whose distance is at least 1.2 times the straight-line distance, with our mean error on them. Every answer of ours is certified and correct. EAR-Oracle: 5 Steiner points per edge.

		ours, uncapped				ours, cap 64		DGG 10^{-3}	NeuroGF		EAR-Oracle	
	obstacles	size	mean	max UB	detour ≥ 1.2	mean	max UB	mean	mean	max	mean	max
TJ	disks 5%	1.77 GB	0.0018%	1.92%	11, 0.0032%	0.016%	4.64%	0.034%	0.49%	9.8%	2.56%	11.2%
	disks 10%	1.53 GB	0.0021%	0.57%	17, 0.0056%	0.021%	1.97%	0.036%	0.49%	8.1%	2.72%	9.65%
	disks 20%	1.33 GB	0.0022%	0.19%	41, 0.0082%	0.018%	2.43%	0.033%	0.50%	20.4%	2.66%	8.00%
	six walls	1.92 GB	0.0028%	0.79%	265, 0.0047%	0.031%	2.76%	0.035%	0.44%	12.4%	2.76%	9.24%
GI	disks 5%	1.80 GB	0.0021%	1.93%	4, 0.0099%	0.023%	5.63%	0.033%	0.50%	19.9%	2.83%	10.1%
	disks 10%	1.54 GB	0.0022%	1.21%	21, 0.0025%	0.023%	3.07%	0.034%	0.51%	8.3%	2.93%	8.81%
	disks 20%	1.34 GB	0.0023%	1.00%	50, 0.0049%	0.020%	1.33%	0.032%	0.53%	23.4%	2.93%	8.74%
	six walls	1.95 GB	0.0022%	0.32%	285, 0.0035%	0.030%	1.41%	0.033%	0.50%	15.9%	2.95%	10.6%

Obstacles. Table 9 repeats the obstacle experiment at 0.5M faces. Our mean error is 0.002–0.003% uncapped and 0.016–0.031% capped, as without obstacles, and stays that low on the pairs whose path has to go around the obstacles: on the walls meshes 265 and 285 of the 2,000 random pairs have a detour of at least 20%, with mean errors of 0.005% and 0.004%. Every answer is certified and correct. DGG keeps its accuracy (0.03%) at 0.2 s per query, and NeuroGF's maximum error grows to 16–23% on some meshes. EAR-Oracle keeps its mean error of 2.6–3.0%, with a maximum of 8–11%, takes 238–318 GB and answers in 0.24–0.42 s, so our uncapped index is 1,000–1,600 times more accurate on average, 180,000–350,000 times faster and 160–180 times smaller (510–560 times with 8-bit codes). The build is where the obstacles cost us. Detours widen the windows, and the walls make our build about 2.5 times as long as the disks (24–25 h uncapped and 19–20 h capped, against 8–11 h and 5.5–9.4 h). EAR-Oracle's build depends less on the obstacles: it takes 11–18 h, so on the disks our build is 1.3–2.2 times faster than EAR's, but on the walls 1.1–1.5 times slower (16.5–17.4 h for EAR).

kNN and range queries in the POI setting. UP-Oracle's own workload asks for distances between points of interest, and we ran kNN ($k = 1, 10, 25$) and range queries (radius 1, 3 and 10% of the chart diagonal) from 20 of them over all others, with their POI files of 50, 500 and 2,500 points on the six terrains at 0.5M faces. The UP-Oracle family has no proximity operator, so it answers every query-object pair. Our index over the POIs takes 1.2, 2.4–2.6 and 6.7–7.3 MB and builds in 4–5 h. Its certified-approximate kNN with $k = 10$ takes 29–73 μs with recall at least 99.5%, and range queries 2–93 μs. UP-Oracle takes 0.7–0.8 ms per kNN query with 50 POIs, 50–72 ms with 500 and 97–122 s with 2,500, where its index is 2.1–2.4 GB and builds in 19–25 h; WSPD-Oracle and RC-Oracle are faster at 2,500 POIs (2.6–5.9 s) but slower at 50 and 500. DGG with an early-stopping Dijkstra takes 20–26 ms at 50 POIs and 0.14–0.20 ms at 2,500. Our exact mode needs sweeps when the POIs are sparse: 0.1–0.9 s at 50 POIs, 0.7–8.9 ms at 500 and 0.06–0.5 ms at 2,500. In the plain POI setting, distances between POIs only, our POI index answers in 0.7–0.9 μs with a mean error of 0.003–0.008%, against UP-Oracle's 0.4–2.3% at 15 μs to 57 ms; but the exact $P \times P$ table built with one VTP sweep per POI stays the best choice up to 2,500 POIs (25 MB, 0.04–3 h), as on the small terrains.

Proximity over many objects. With 10,000 random objects at 0.5M faces, certified-approximate kNN answers in 23–155 μs for $k = 1$ and 60–169 μs for $k = 10$ with recall of at least 98.9%. The exact mode needs a sweep only for the candidates the intervals cannot separate, in 0–2% of the queries for $k = 1$ and 13–22% for $k = 10$ (0.3–0.5 objects per query on average): 24–158 μs for $k = 1$ and 0.3–0.5 ms for $k = 10$. An on-the-fly search from the query point that stops once the k nearest objects are settled, pruned by chart distances, takes 1.2–1.9 ms and 3.8–7.0 ms, and a full sweep 2.7–3.2 s. At 2.5M faces, $k = 10$ takes 0.4 ms approximately and 2.5 ms exactly, against an 11 s sweep.

The small terrains on the cluster. Rerunning the small-terrain configurations of Table 2 on the cluster gives the same picture: on Gunnison10k with $\varepsilon = 0.1$ the build takes 194 s (202 s on the laptop) and the mean error is 0.0076%.

7 Discussion

The experiments support the main claim: a terrain distance oracle can answer in about a microsecond and certify every answer, with errors far below those of the Steiner-graph oracles and an index two to three orders of magnitude smaller, up to terrains with millions of faces. The certificate is what makes the index usable as a database operator. A kNN or range query can decide most candidates from the intervals alone, and a user who needs a guaranteed answer can ask for UB, which is the length of an actual path, or for an exact refinement only when the interval is too wide. DGG reaches a similar accuracy with a much shorter build, but searches per query. Table 10 sums up which method to use when, from our measurements.

7.1 Limitations

Asymptotic size. The uncapped index stores $\Theta(n^{1.5})$ labels, as the top cuts have $\Theta(\sqrt{n})$ portals and every vertex stores them. The capped variant is near-linear only for bounded slope, detour and number of runs, its accuracy relies on an exact refine that our analysis does not confine to close pairs, and Djidjev and Sommer's oracle is asymptotically smaller in theory. In floating point the guarantees are validated, not proved (Section 4).

Table 10: Which method to use, from the measurements of Section 6. Break-even counts at 0.5–5M faces include our build.

situation	method
a few queries, terrain not queried again	exact VTP sweep, or DGG for a few hundred (no bound)
distances among a fixed set of up to ~2,500 POIs	exact $P \times P$ table, one sweep per POI
10^3–10^5 queries, no bound needed	DGG: builds in minutes, 0.3–7 s per query
beyond 590–19,600 queries (vs. VTP) or 5,700–106,000 (vs. DGG)	CCI: microsecond answers
a bound on each answer, or exact kNN and range	CCI, with refine for undecided pairs
memory-bound index	CCI with cap 64: near-linear size
terrain that changes	on-the-fly sweeps; UP-Oracle for POIs

Build. The build is dominated by one exact sweep per portal. Uncapped it grows like $n^{1.5}$, takes 96–110 h on one thread at 2M faces and 110–125 h at 2.5M faces (15.7 h on 32 threads); capped it takes 51–52 h at 2.5M faces. Paths that detour around walls make it 2.5 times longer. The build is the price of the certificate and of microsecond queries, and it is recovered after roughly 600–106,000 queries, depending on the terrain size, the variant and the alternative.

Slopes and weights. λ enters the near radius, the window and the ellipse, and a small $w_{\min}$ widens all three. Steep slopes raise σ_C and lengthen the capped tails. On weighted terrains we certify only the upper bound.

Static terrains. The index is static. Terrains that change, the setting of UP-Oracle, would need the affected cuts rebuilt.

8 Conclusion

The Certified Cut Index answers terrain distance queries in about a microsecond with an interval within a factor $1 \pm \varepsilon$ of the exact surface distance, so kNN and range queries are decided from the intervals and refined only where needed. Future work includes validated enclosures for the solvers, a faster build, and updates for changing terrains.

AI use statement

We used generative AI tools to help write and test parts of the implementation and the experiment scripts, to summarise related literature, and to draft and edit parts of this manuscript. The author reviewed the code, checked the cited works against the original sources and verified the proofs and the experimental results, and takes full responsibility for the content.

References

[1] Ittai Abraham, Daniel Delling, Andrew V. Goldberg, and Renato F. Werneck. 2011. A Hub-Based Labeling Algorithm for Shortest Paths in Road Networks. In *Experimental Algorithms (SEA)*. 230–241.

[2] Yohanes Yudhi Adikusuma, Zheng Fang, and Ying He. 2020. Fast Construction of Discrete Geodesic Graphs. *ACM Transactions on Graphics* 39, 2 (2020), 14:1–14:14.

[3] Takuya Akiba, Yoichi Iwata, and Yuichi Yoshida. 2013. Fast Exact Shortest-Path Distance Queries on Large Networks by Pruned Landmark Labeling. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 349–360.

[4] Lyudmil Aleksandrov, Hristo N. Djidjev, Hua Guo, Anil Maheshwari, Doron Nussbaum, and Jörg-Rüdiger Sack. 2010. Algorithms for Approximate Shortest Path Queries on Weighted Polyhedral Surfaces. *Discrete & Computational Geometry* 44, 4 (2010), 762–801.

[5] Lyudmil Aleksandrov, Anil Maheshwari, and Jörg-Rüdiger Sack. 2005. Determining Approximate Shortest Paths on Weighted Polyhedral Surfaces. *J. ACM* 52, 1 (2005), 25–53.

[6] Jon Louis Bentley. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM* 18, 9 (1975), 509–517.

[7] Oliver Burghard and Reinhard Klein. 2015. Simple, Robust, Constant-Time Bounds on Surface Geodesic Distances using Point Landmarks. In *Vision, Modeling and Visualization (VMV)*. 17–24.

[8] Jindong Chen and Yijie Han. 1990. Shortest Paths on a Polyhedron. In *Proceedings of the Sixth Annual Symposium on Computational Geometry (SoCG)*. 360–369.

[9] Keenan Crane, Clarisse Weischedel, and Max Wardetzky. 2013. Geodesics in Heat: A New Approach to Computing Distance Based on Heat Flow. *ACM Transactions on Graphics* 32, 5 (2013), 152:1–152:11.

[10] Ke Deng, Xiaofang Zhou, Heng Tao Shen, Kai Xu, and Xuemin Lin. 2006. Surface k-NN Query Processing. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*. 78.

[11] Hristo N. Djidjev and Christian Sommer. 2011. Approximate Distance Queries for Weighted Polyhedral Surfaces. In *Algorithms – ESA 2011 (Lecture Notes in Computer Science, Vol. 6942)*. 579–590.

[12] Muhammad Farhan, Henning Koehler, Robert Ohsaka, and Qing Wang. 2023. Hierarchical Cut Labelling – Scaling Up Distance Queries on Road Networks. *Proceedings of the ACM on Management of Data* 1, 4 (2023), 244:1–244:25.

[13] Cyril Gavoille, David Peleg, Stéphane Pérennes, and Ran Raz. 2004. Distance Labeling in Graphs. *Journal of Algorithms* 53, 1 (2004), 85–112.

[14] Andrew V. Goldberg and Chris Harrelson. 2005. Computing the Shortest Path: A* Search Meets Graph Theory. In *Proceedings of the 16th ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 156–165.

[15] Craig Gotsman and Kai Hormann. 2022. Compressing Geodesic Information for Fast Point-to-Point Geodesic Distance Queries. In *SIGGRAPH Asia 2022 Conference Papers*. 4:1–4:9.

[16] Bo Huang, Victor Junqiu Wei, Raymond Chi-Wing Wong, and Bo Tang. 2023. EAR-Oracle: On Efficient Indexing for Distance Queries between Arbitrary Points on Terrain Surface. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 1 (2023).

[17] Manohar Kaul, Raymond Chi-Wing Wong, and Christian S. Jensen. 2015. New Lower and Upper Bounds for Shortest Distance Queries on Terrains. *Proceedings of the VLDB Endowment* 9, 3 (2015), 168–179.

[18] Manohar Kaul, Raymond Chi-Wing Wong, Bin Yang, and Christian S. Jensen. 2013. Finding Shortest Paths on Terrains by Killing Two Birds with One Stone. *Proceedings of the VLDB Endowment* 7, 1 (2013), 73–84.

[19] Ken-ichi Kawarabayashi, Philip N. Klein, and Christian Sommer. 2011. Linear-Space Approximate Distance Oracles for Planar, Bounded-Genus and Minor-Free Graphs. In *Automata, Languages and Programming (ICALP)*. 135–146.

[20] Ron Kimmel and James A. Sethian. 1998. Computing Geodesic Paths on Manifolds. *Proceedings of the National Academy of Sciences* 95, 15 (1998), 8431–8435.

[21] Philip N. Klein. 2002. Preprocessing an Undirected Planar Network to Enable Fast Approximate Distance Queries. In *Proceedings of the Thirteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 820–827.

[22] Richard J. Lipton and Robert Endre Tarjan. 1979. A Separator Theorem for Planar Graphs. *SIAM J. Appl. Math.* 36, 2 (1979), 177–189.

[23] Joseph S. B. Mitchell, David M. Mount, and Christos H. Papadimitriou. 1987. The Discrete Geodesic Problem. *SIAM J. Comput.* 16, 4 (1987), 647–668.

[24] Dian Ouyang, Lu Qin, Lijun Chang, Xuemin Lin, Ying Zhang, and Qing Zhu. 2018. When Hierarchy Meets 2-Hop-Labeling: Efficient Shortest Distance Queries on Road Networks. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. 709–724.

[25] Bo Pang, Zhongtian Zheng, Guoping Wang, and Peng-Shuai Wang. 2023. Learning the Geodesic Embedding with Graph Neural Networks. *ACM Transactions on Graphics* 42, 6 (2023), 191:1–191:12.

[26] Yipeng Qin, Xiaoguang Han, Hongchuan Yu, Yizhou Yu, and Jianjun Zhang. 2016. Fast and Exact Discrete Geodesic Computation Based on Triangle-Oriented Wavefront Propagation. *ACM Transactions on Graphics* 35, 4 (2016), 125:1–125:13.

[27] Cyrus Shahabi, Lu-An Tang, and Songhua Xing. 2008. Indexing Land Surface for Efficient kNN Query. *Proceedings of the VLDB Endowment* 1, 1 (2008), 1020–1031.

[28] Vitaly Surazhsky, Tatiana Surazhsky, Danil Kirsanov, Steven J. Gortler, and Hugues Hoppe. 2005. Fast Exact and Approximate Geodesics on Meshes. *ACM Transactions on Graphics* 24, 3 (2005), 553–560.

[29] Mikkel Thorup. 2004. Compact Oracles for Reachability and Approximate Distances in Planar Digraphs. *J. ACM* 51, 6 (2004), 993–1024.

[30] Mikkel Thorup. 2007. Compact Oracles for Approximate Distances Around Obstacles in the Plane. In *Proceedings of the 15th European Symposium on Algorithms (ESA) (LNCS, Vol. 4698)*. 383–394.

[31] Jacobus van der Laan, Frank Staals, and Lorenzo Theunissen. 2026. Approximate Dynamic Nearest Neighbor Searching in a Polygonal Domain. arXiv:2603.11775.
[32] Victor Junqiu Wei, Raymond Chi-Wing Wong, Cheng Long, and David M. Mount. 2017. Distance Oracle on Terrain Surface. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. 1211–1226.
[33] Shi-Qing Xin and Guo-Jin Wang. 2009. Improving Chen and Han's Algorithm on the Discrete Geodesic Problem. *ACM Transactions on Graphics* 28, 4 (2009), 104:1–104:8.
[34] Shi-Qing Xin, Xiang Ying, and Ying He. 2012. Constant-Time All-Pairs Geodesic Distance Query on Triangle Meshes. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games (I3D)*. 31–38.
[35] Yinzhao Yan and Raymond Chi-Wing Wong. 2024. Proximity Queries on Point Clouds using Rapid Construction Path Oracle. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 1 (2024).
[36] Yinzhao Yan, Raymond Chi-Wing Wong, and Christian S. Jensen. 2025. An Efficiently Updatable Path Oracle for Terrain Surfaces. *IEEE Transactions on Knowledge and Data Engineering* 37 (2025).
[37] Xiang Ying, Xiaoning Wang, and Ying He. 2013. Saddle Vertex Graph (SVG): A Novel Solution to the Discrete Geodesic Problem. *ACM Transactions on Graphics* 32, 6 (2013), 170:1–170:12.
[38] Qijian Zhang, Junhui Hou, Yohanes Yudhi Adikusuma, Wenping Wang, and Ying He. 2023. NeuroGF: A Neural Representation for Fast Geodesic Distance and Path Queries. In *Advances in Neural Information Processing Systems (NeurIPS)*.

A Proofs

We use the notation of Section 4. In the implementation a portal that falls within 10^{-7} of a face edge is moved along the line by about 10^{-5} edge lengths before it is inserted, so a run end may not be a portal. It then leaves an end stretch [run end, p_k] of arc e_k, and the lower bound also takes the term $f_k - 2e_k$.

Lemma A.1 (crossing). *Every curve on T between two points on opposite sides of ℓ_C meets ℓ_C.*

Proof. The a-coordinate of its chart projection is continuous, below v at one end and above at the other. □

Proof of Lemma 4.3 with end stretches. If the crossing x^* lies in an end stretch with portal p_k, then $f_k \le d(s,x^*)+2A(x^*,p_k)+d(x^*,t) \le d(s,t)+2e_k$, so $f_k - 2e_k \le d(s,t)$. The interval case is as in the main text. □

Proof of Theorem 4.4. Let $|s-t| > R$ and $d = d(s,t)$, so $d \ge \lambda|s-t| > g/\varepsilon$ by (1). Let x^* be the crossing of a shortest path in the interval $[p_j, p_{j+1}]$ with arc gap A_j, and $a = A(p_j, x^*)$.

Upper bound. $\min(f_j, f_{j+1}) \le d + \min(2a, 2(A_j - a)) \le d + A_j$. If the interval lies in I_C then $A_j \le g < \varepsilon d$. Otherwise every point of the interval, x^* included, has offset at least c_j from I_C, so $d \ge 2\lambda c_j$ by Lemma 4.1, and (2) gives $A_j \le \max(g, 2\varepsilon\lambda c_j) \le \varepsilon d$. In an end stretch, $f_k \le d + 2e_k$, and $2e_k$ is far below $g < \varepsilon d$.

Lower bound. Every $f \ge d$, so an interval term satisfies $\frac{1}{2}(f_j + f_{j+1}) - A_j \ge d - A_j$, which is at least $(1-\varepsilon)d$ if $A_j \le \varepsilon d$. Otherwise $A_j > \varepsilon d > g$, so the interval is a tail interval with $A_j \le 2\varepsilon\lambda c_j$, hence $c_j > d/(2\lambda)$. Both of its portals p have offset at least c_j from B_M, so by (1) $f(p) \ge \lambda(|s-p| + |p-t|) \ge 2\lambda c_j$, and the term is at least $2\lambda c_j(1-\varepsilon) > (1-\varepsilon)d$. An end-stretch term is $f_k - 2e_k \ge d - 2e_k > (1-\varepsilon)d$. The chord term is a lower bound, so $\mathrm{LB} \ge (1-\varepsilon)d$. □

Proof of Theorem 4.5. Lemma 4.3 holds for any portal set, so $\mathrm{LB} \le d \le \mathrm{UB}$. The query accepts an answer only if $\mathrm{UB} \le (1+\varepsilon)\mathrm{LB}$, and computes every other pair exactly. A refined pair is computed by one exact sweep on the faces meeting $\mathrm{bbox}(s,t)$ dilated by $\mathrm{UB}/(2\lambda)$, which contains every path of length at most UB by Lemma 4.2, so it is exact.

For the size, a run whose part inside I_C has cost a gets at most $a/g_C + 2$ portals there, plus its two ends, and $g_C \ge A(I_C)/k$, so the box holds at most $k + O(r_C)$ portals. Cost and chart length along the cut satisfy $\lambda\, dc \le dA \le \sigma_C\, dc$. In a tail the gap is g_C while $2\varepsilon\lambda c < g_C$, a chart length of $g_C/(2\varepsilon\lambda)$ and so at most $\sigma_C/(2\varepsilon\lambda) + O(r_C)$ portals. From then on a step of cost $2\varepsilon\lambda c$ advances the chart offset by at least $2\varepsilon\lambda c/\sigma_C$, so the offset grows by a factor at least $1 + 2\varepsilon\lambda/\sigma_C$ per portal, from $g_C/(2\varepsilon\lambda)$ up to the window length μ_C/λ. Since $g_C \ge A(I_C)/k \ge \lambda|I_C|/k$, the ratio of the two offsets is at most $2\varepsilon\mu_C/g_C \le 2\varepsilon k\kappa_C$, and as $2\varepsilon\lambda/\sigma_C \le 2$, this takes at most $\frac{\sigma_C}{\varepsilon\lambda}\log^+(2\varepsilon k\kappa_C) + O(r_C)$ portals. A vertex stores the rows of the cuts on its path to the root, which gives the label count, and its near table holds $|N_R(u)|$ entries. If σ_C/λ, κ_C and r_C are bounded, a cut has $k + O(\varepsilon^{-1}\log k)$ portals, and median cuts give a hierarchy of depth $O(\log n)$. □

Proof of Theorem 4.6. $\mathrm{UB}(p,q)$ is the cost of a path from p to a corner, then a path certified by $\mathrm{UB}(a_i,b_j)$, then to q. For the lower bound, $d(a_i,b_j) \le d(a_i,p) + d(p,q) + d(q,b_j) \le w_f|P - A_i| + d(p,q) + w_{f'}|Q - B_j|$, so every term of the maximum is at most $d(p,q)$. Pairs whose interval is wider than $1+2\varepsilon$, and near pairs, are computed exactly on an ellipse sub-mesh with both points inserted as vertices. □